



数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期

第3章 栈和队列

提纲

CONTENTS



3.1 栈

3.2 队列

3.1 栈

3.1.1 栈的定义

栈是一种只能在一端进行插入或删除操作的线性表。



栈只能选取同一个端点进行插入和删除操作

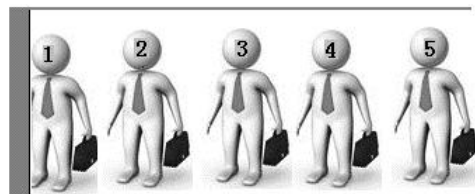


3.1 栈

3.1.1 栈的定义

栈的主要特点是“**后进先出**”，即后进栈的元素先出栈。栈也称为**后进先出表**。

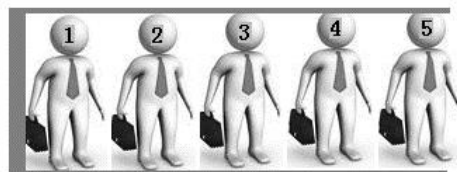
例如：



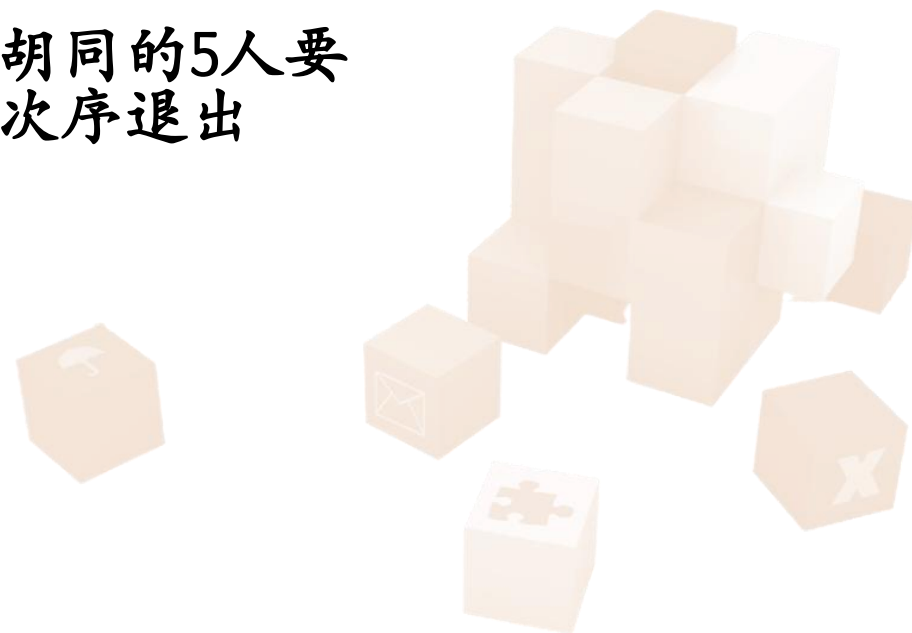
假设死胡同的宽度
恰好只够一个人



走进死胡同的5人要
按相反次序退出



死胡同就是一个栈！

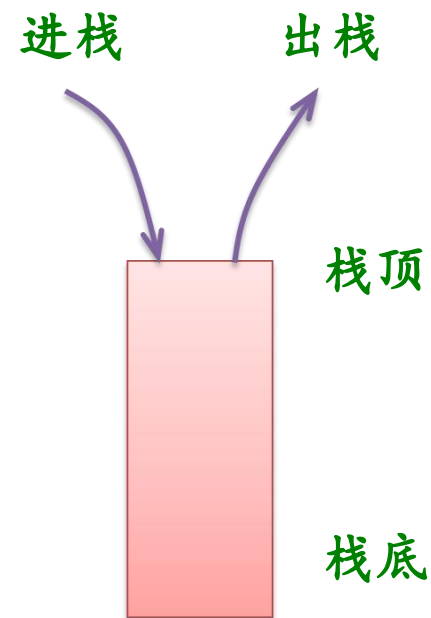


3.1 栈

3.1.1 栈的定义

栈的几个概念

- 允许进行插入、删除操作的一端称为**栈顶**。
- 表的另一端称为**栈底**。
- 当栈中没有数据元素时，称为**空栈**。
- 栈的插入操作通常称为**进栈**或**入栈**。
- 栈的删除操作通常称为**退栈**或**出栈**。



栈示意图

3.1 栈

3.1.1 栈的定义

示例 设一个栈的输入序列为 a, b, c, d , 则借助一个栈所得到的输出序列不可能是 ()。

A. c, d, b, a

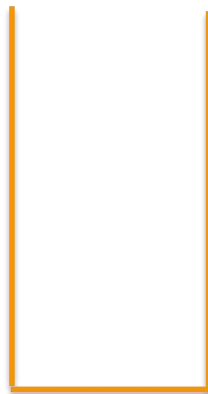
B. d, c, b, a

C. a, c, d, b

D. d, a, b, c

选项D是不可能的?

$d \ c \ b \ a$



栈

下一步不可能出栈 a

答案为D



3.1 栈

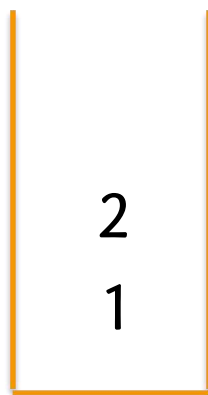
3.1.1 栈的定义

【例3.3】 一个栈的入栈序列为1, 2, 3, $\dots$, n , 其出栈序列是 $p_1, p_2, p_3, \dots, p_n$ 。若 $p_1=3$, 则 p_2 可能取值的个数是多少?

A. $n-3$ B. $n-2$ C. $n-1$ D. 无法确定

1、2、3进栈， 3出栈的结果：

$n \dots 4$



栈



不可能是1和3

共有 $n-2$ 可能

答案为B

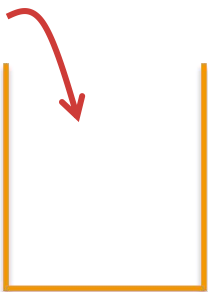
3.1 栈

示例 一个栈的入栈序列为 $1, 2, 3, \dots, n$ ，其出栈序列是 $p_1, p_2, p_3, \dots, p_n$ 。若 $p_2=3$ ，则 p_3 可能取值的个数是 () 多少？(全国考研题)

A. $n-3$ B. $n-2$ **C. $n-1$** D. 无法确定

$n \dots 3 2 1$

解



不可能是3

p_3 可以取1: 1进, 2进, 2出, 3进, **3出**, 1出, $\dots$ 。

p_3 可以取2: 1进, 1出, 2进, 3进, **3出**, 2出, $\dots$ 。

p_3 可以取4: 1进, 1出, 2进, 3进, **3出**, 4进, 4出, $\dots$ 。

p_3 可以取5: 1进, 1出, 2进, 3进, **3出**, 4进, 5进, 5出, $\dots$ 。

$\dots$

p_3 可以取除了3外的任何值, 共有 $n-1$ 种可能。答案为 **C**。

3.1 栈

3.1.1 栈的定义

栈抽象数据类型 = 逻辑结构 + 基本运算 (运算描述)

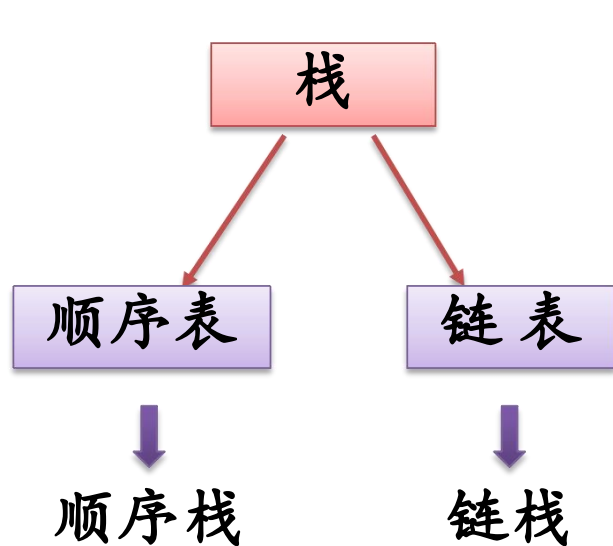
栈的几种基本运算如下:

- ① InitStack(&s): 初始化栈。构造一个空栈s。
- ② DestroyStack(&s): 销毁栈。释放栈s占用的存储空间。
- ③ StackEmpty(s): 判断栈是否为空:若栈s为空, 则返回真; 否则返回假。
- ④ Push(&S, e): 进栈。将元素e插入到栈s中作为栈顶元素。
- ⑤ Pop(&s, &e): 出栈。从栈s中退出栈顶元素, 并将其值赋给e。
- ⑥ GetTop(s, &e): 取栈顶元素。返回当前的栈顶元素, 并将其值赋给e。

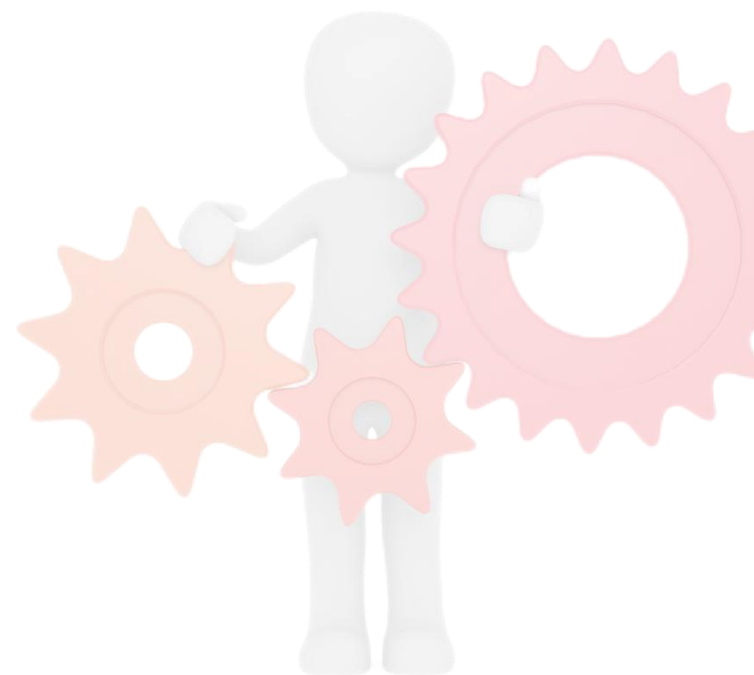
3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

栈中元素逻辑关系与线性表的相同，栈可以采用与线性表相同的存储结构。



逻辑结构
↓
存储结构



3.1 栈

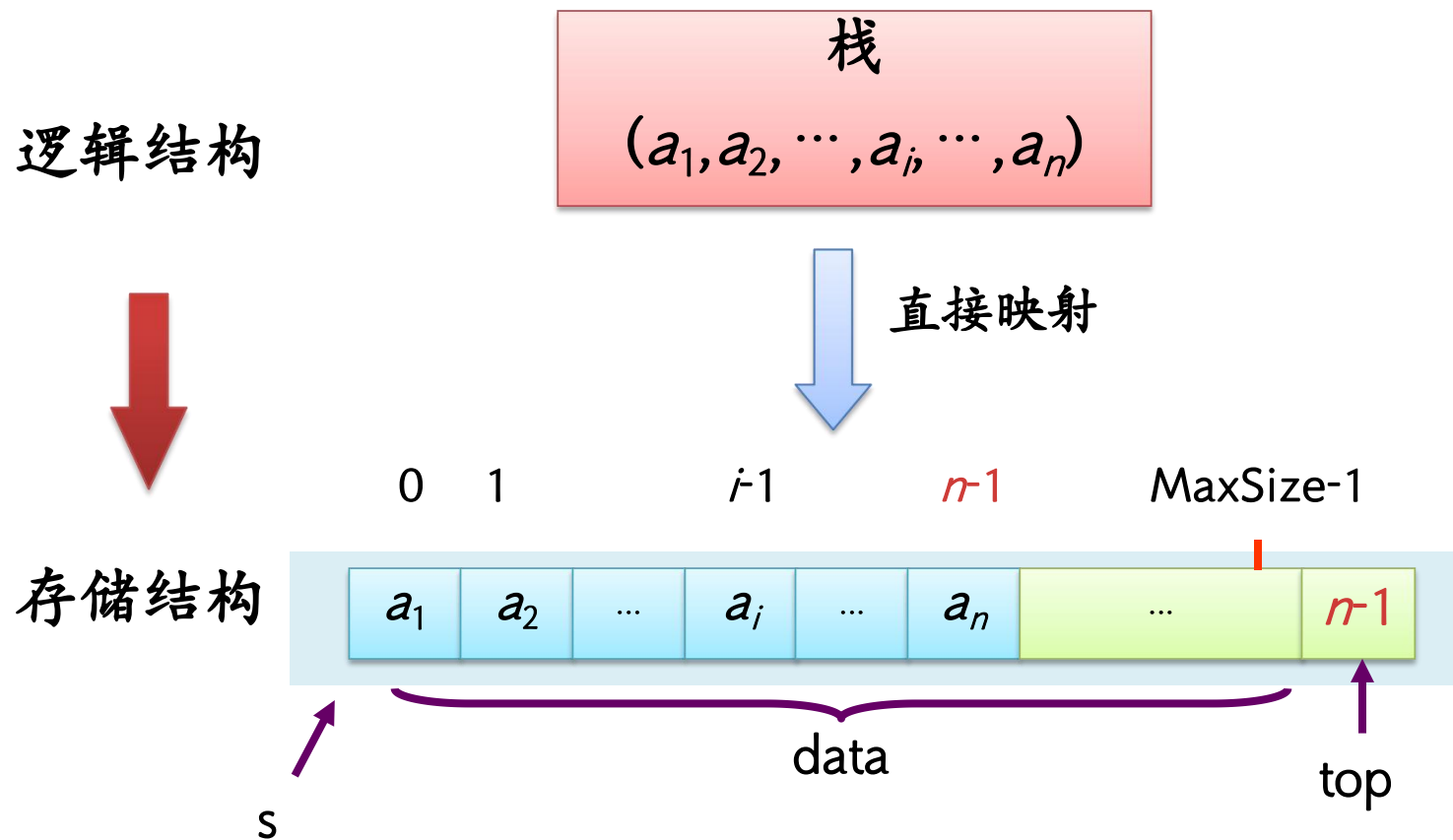
3.1.2 栈的顺序存储结构及其基本运算实现

假设栈的元素个数最大不超过正整数MaxSize，所有的元素都具有同一数据类型ElemType，则可用下列方式来声明**顺序栈**类型SqStack:

```
typedef struct
{ ElemType data[MaxSize];
  int top;           //栈顶指针
} SqStack;
```

3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

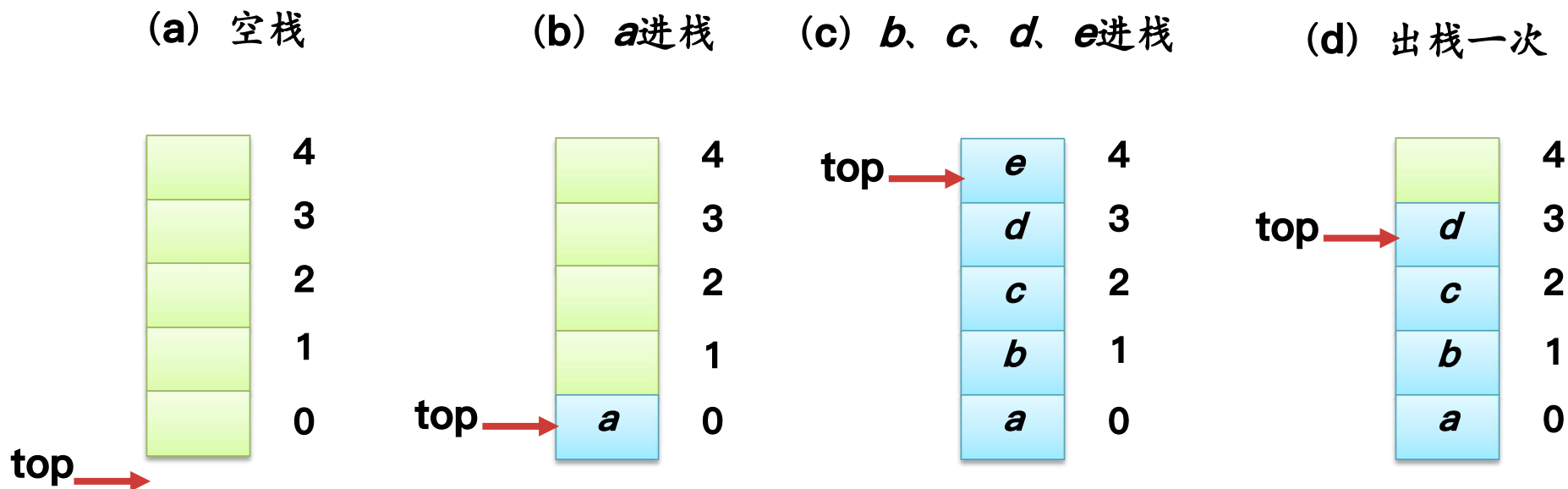


顺序栈的示意图

3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

例如: **MaxSize=5**



总结:

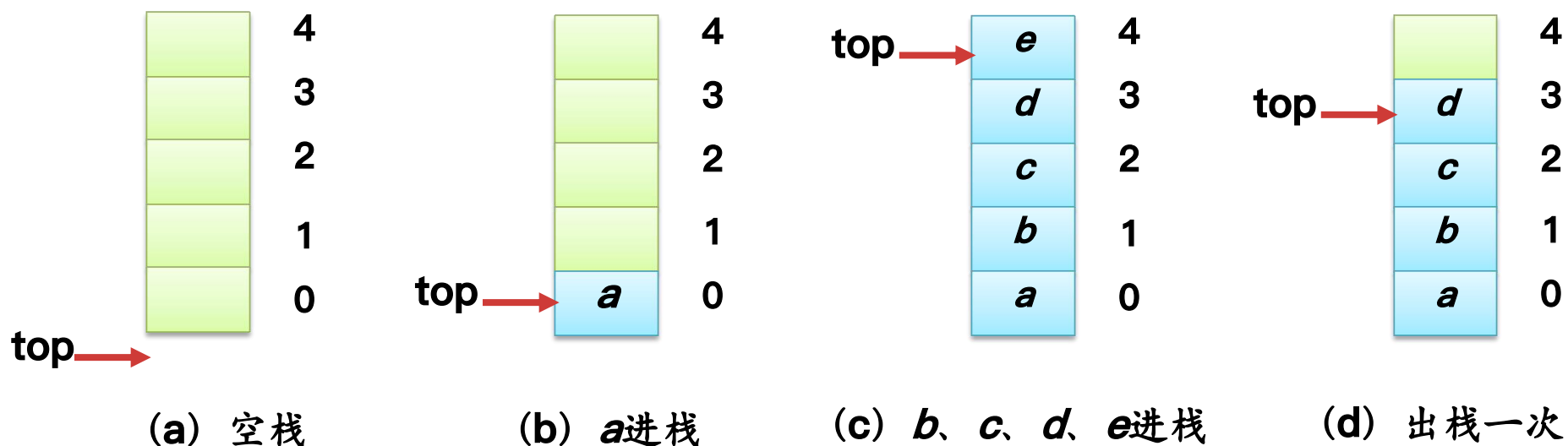
- 约定 **top** 总是指向栈顶元素, 初始值为 -1
- 当 $\text{top} = \text{MaxSize} - 1$ 时不能再进栈 — **栈满**
- 进栈时 **top** 增 1 ($\text{top}++$), 出栈时 **top** 减 1 ($\text{top}--$)



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

顺序栈的各种状态



顺序栈4要素:

- 栈空条件: $s \rightarrow \text{top} == -1$
- 栈满条件: $s \rightarrow \text{top} == \text{MaxSize} - 1$
- 进栈e操作: $\text{top}++$; 将e放在top处
- 退栈操作: 从top处取出元素e; $\text{top}--$;

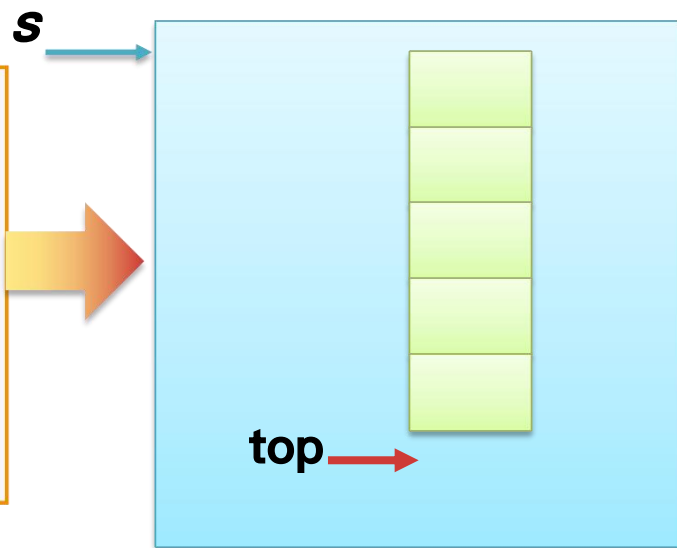
3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

(1) 初始化栈InitStack(&s)

建立一个新的空栈s，实际上是将栈顶指针指向-1即可。

```
void InitStack(SqStack *&s)
{ s=(SqStack *)malloc(sizeof(SqStack));
  s->top=-1;
}
```



注意： s为栈指针，top为s所指栈的栈顶指针

3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

(2) 销毁栈DestroyStack(&s)

释放栈s占用的存储空间。

```
void DestroyStack(SqStack *&s)
{
    free(s);
}
```



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

(3) 判断栈是否为空 StackEmpty(s)

栈S为空的条件是 $s \rightarrow \text{top} == -1$ 。

```
bool StackEmpty(SqStack *s)
{
    return(s->top==-1);
}
```



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

(4) 进栈Push(&s, e)

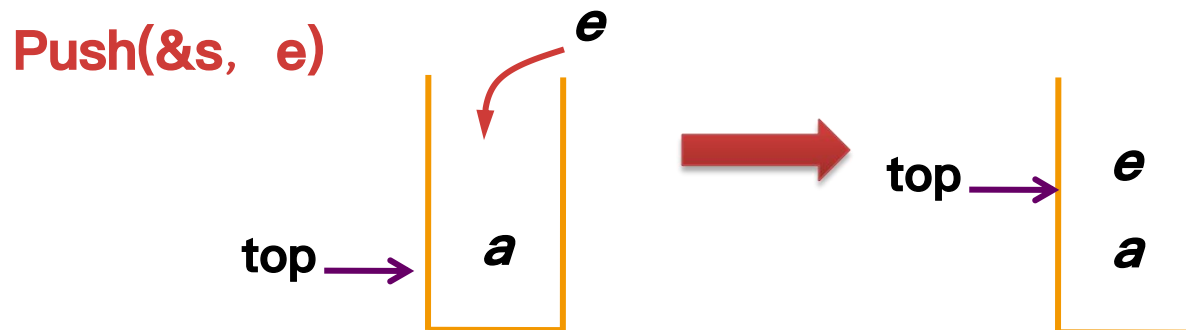
在栈不满的条件下，先将栈指针增1，然后在该位置上插入元素e。

```
bool Push(SqStack *&s, ElemType e)
{ if (s->top==MaxSize-1)           //栈满的情况，即栈上溢出
    return false;
    s->top++;                        //栈顶指针增1
    s->data[s->top]=e;               //元素e放在栈顶指针处
    return true;
}
```

先++
后存放

不能写成s->data[top]=e

因为top跟data一样，是属于s所指向的结构体，需要通过s才能访问。



3.1 栈

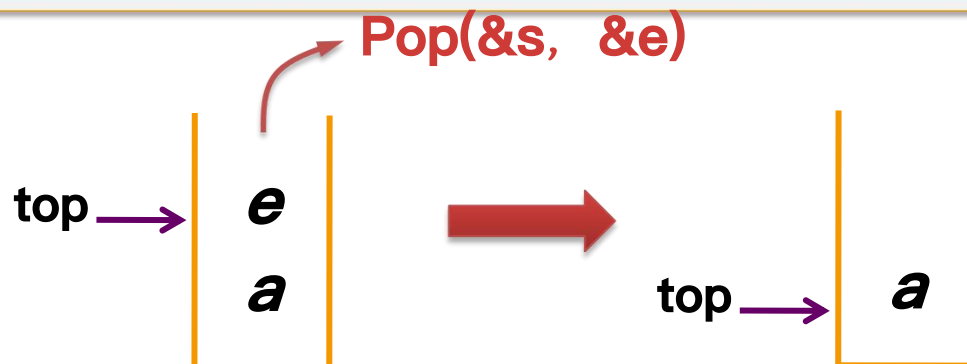
3.1.2 栈的顺序存储结构及其基本运算实现

(5) 出栈Pop(&s, &e)

在栈不为空的条件下，先将栈顶元素赋给e，然后将栈指针减1。

```
bool Pop(SqStack *&s, ElemType &e)
{ if (s->top == -1)           //栈为空的情况，即栈下溢出
    return false;
    e = s->data[s->top];       //取栈顶指针元素的元素
    s->top--;                  //栈顶指针减1
    return true;
}
```

先取值
后++



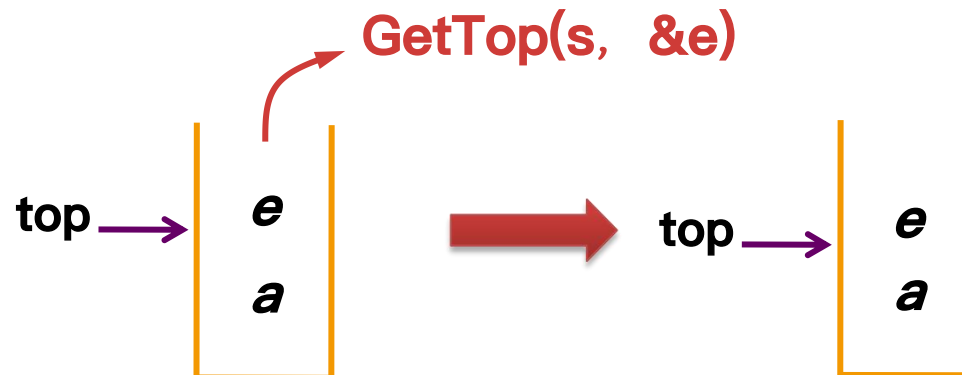
3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

(6) 取栈顶元素 `GetTop(s, &e)`

在栈不为空的条件下，将栈顶元素赋给 e 。

```
bool GetTop(SqStack *s, ElemType &e)
{  if (s->top == -1)                // 栈为空的情况，即栈下溢出
    return false;
    e = s->data[s->top];  // 取栈顶指针元素的元素
    return true;
}
```



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

顺序栈的应用算法设计

【例3.4】 设计一个算法利用顺序栈判断一个字符串是否是对称串。

所谓**对称串**是指从左向右读和从右向左读的序列相同。

算法设计思路

- 字符串str的所有元素依次进栈，产生的连续出栈序列正好与str的顺序相反。
- 如果str与连续出栈序列相同，则是**对称串**。



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

例如, $\text{str} = \text{"abcba"}$



str 进栈并连续出栈序列 $\text{str}' = \text{"abcba"}$



$\text{str} = \text{str}'$, str 是**对称串**



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

```
bool symmetry(ElemType str[])
{ int i; ElemType e;
  SqStack *st;
  InitStack(st);                                //初始化栈

  for (i=0;str[i]!='\0';i++)                    //将串所有元素进栈
    Push(st, str[i]);                          //元素进栈

  for (i=0;str[i]!='\0';i++)
  { Pop(st, e);                                //退栈元素e
    if (str[i]!=e)                             //若e与当前串元素不同则不是对称串
    { DestroyStack(st);                        //销毁栈
      return false;
    }
  }

  DestroyStack(st);                             //销毁栈
  return true;
}
```

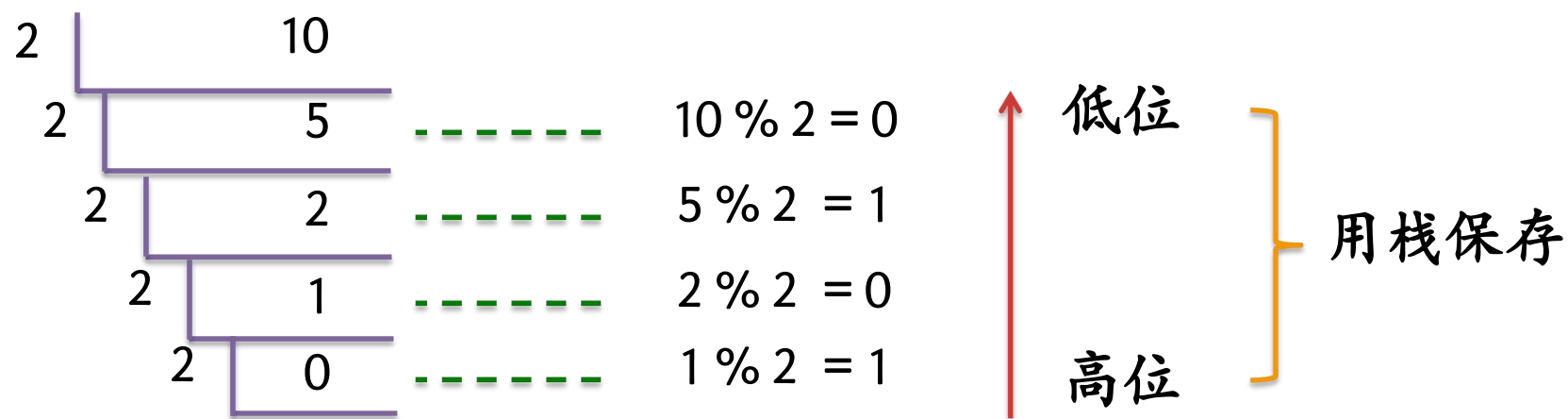
3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

示例

设计一个算法，利用栈将一个正十进制整数转换为二进制数并输出。

十进制 $\Rightarrow$ 二进制: $n=10$



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

```
#include "sqstack1.cpp"    //包含整数栈的基本运算函数
void trans(int n)
{
    int e;
    SqStack *st;
    InitStack(st);
    while (n>0)
    { Push(st,n%2);
      n=n/2;
    }
    while (!StackEmpty(st)) //输出对应的二进制数
    { Pop(st,e);
      printf("%d",e);
    }
    printf("\n");
    DestroyStack(st);
}
```

3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

共享栈

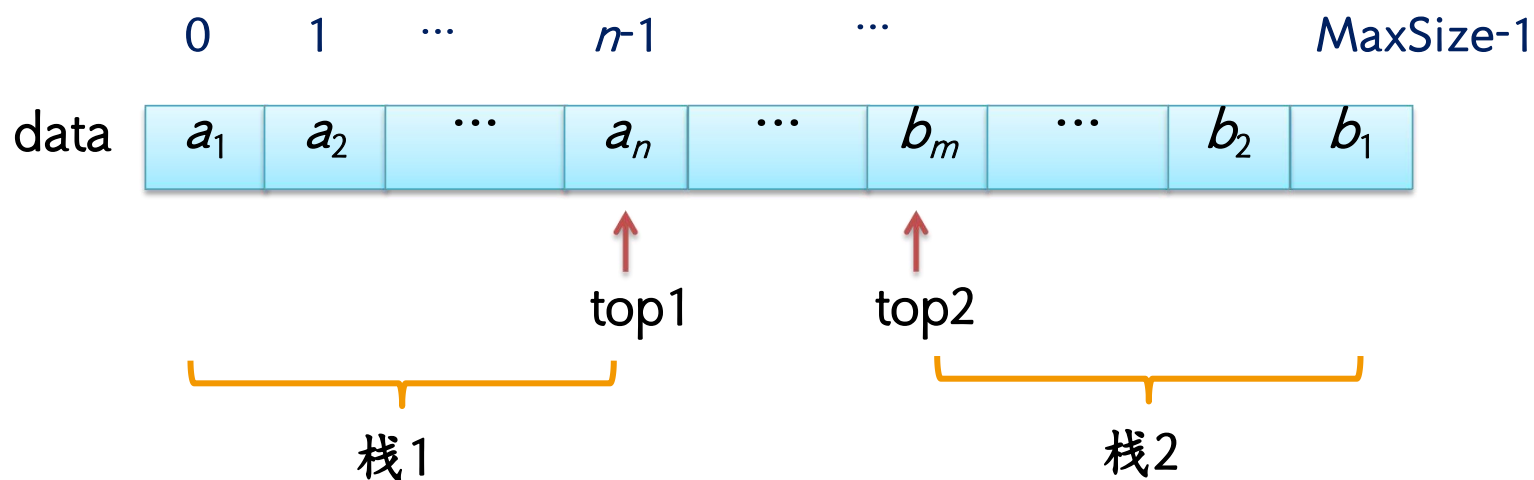
若需要用到两个相同类型的栈，可用一个数组 `data[0..MaxSize-1]` 来实现这两个栈，这称为 **共享栈**。



3.1 栈

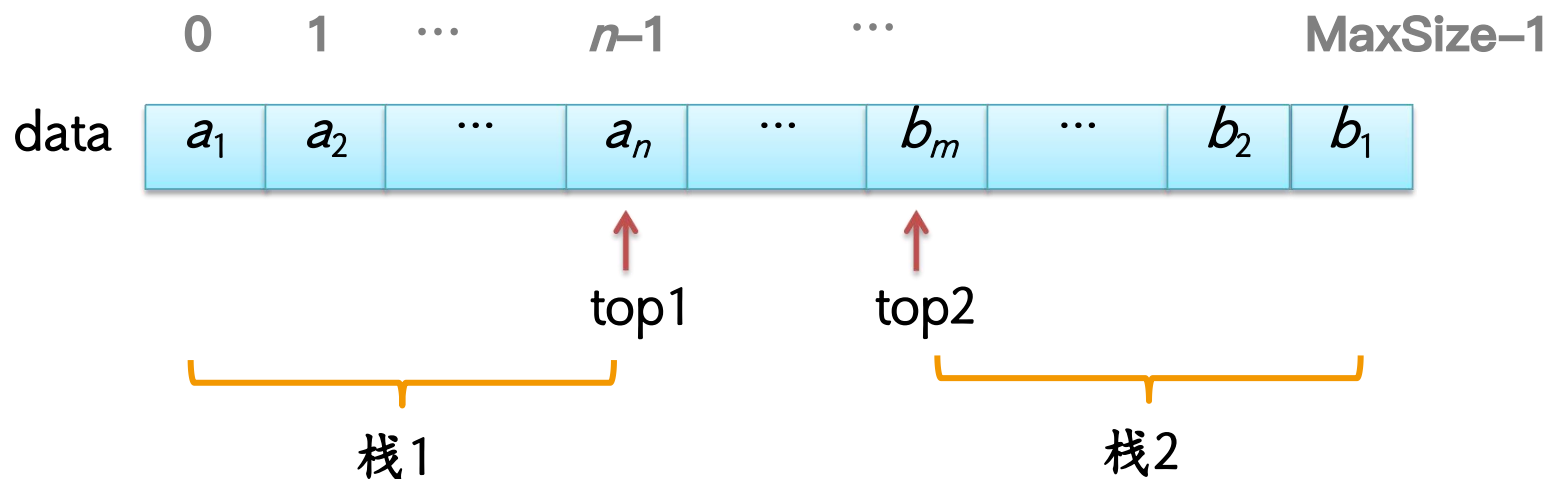
3.1.2 栈的顺序存储结构及其基本运算实现

栈的一端不动，元素向另外一端伸展



3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现

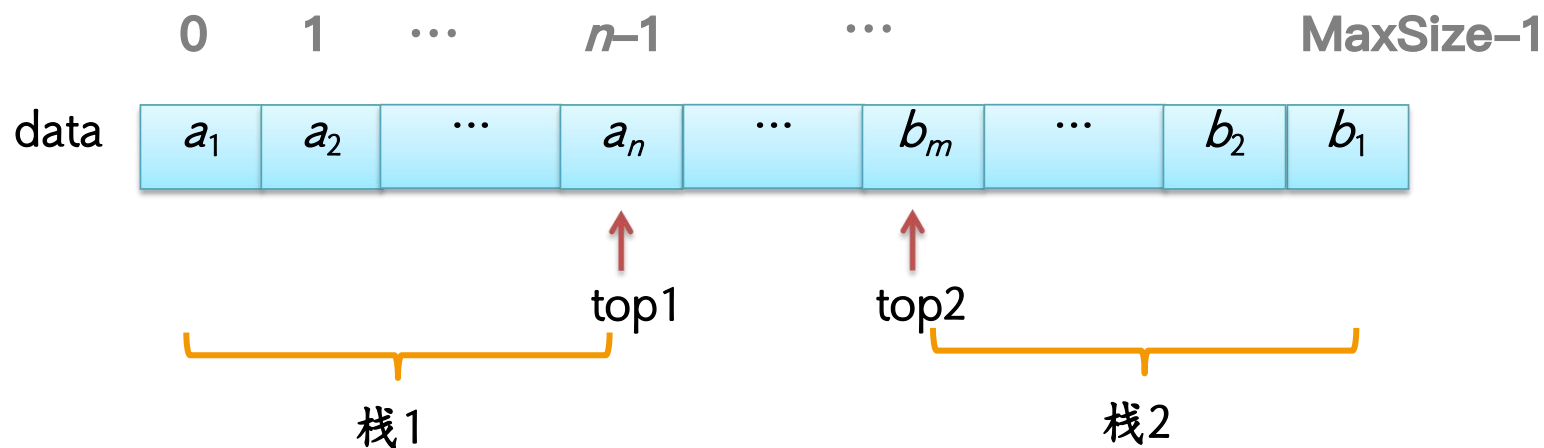


共享栈类型:

```
typedef struct
{
    ElemType data[MaxSize];    //存放共享栈中元素
    int top1, top2;           //两个栈的栈顶指针
} DStack;
```

3.1 栈

3.1.2 栈的顺序存储结构及其基本运算实现



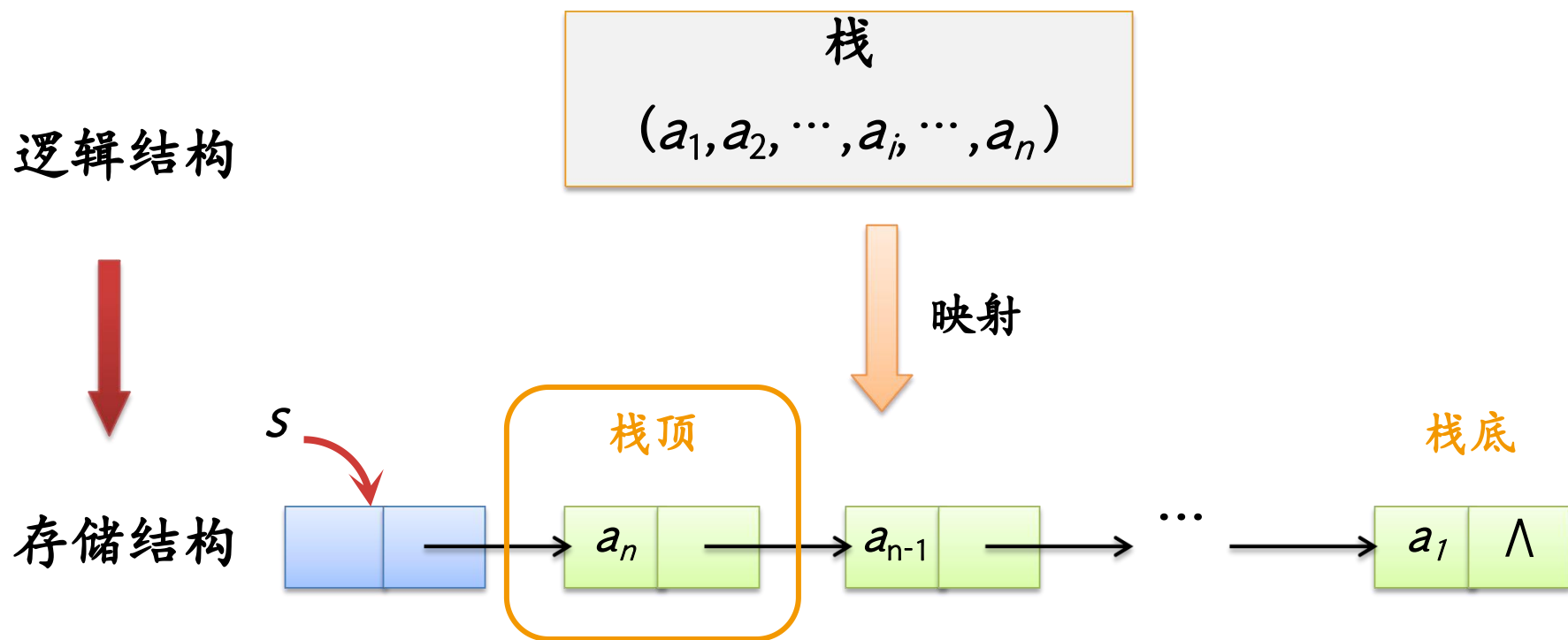
共享栈的4要素:

- 栈空条件, 栈1空: `top1 == -1`; 栈2空: `top2 == MaxSize`
- 栈满条件: `top1 == top2 - 1`
- 元素`x`进栈操作, 进栈1操作: `top1++`; `data[top1] = x`; 进栈2操作: `top2--`; `data[top2] = x`;
- 出栈`x`操作, 出栈1操作, `x = data[top1]`; `top1--`; 出栈2操作, `x = data[top2]`; `top2++`;

3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

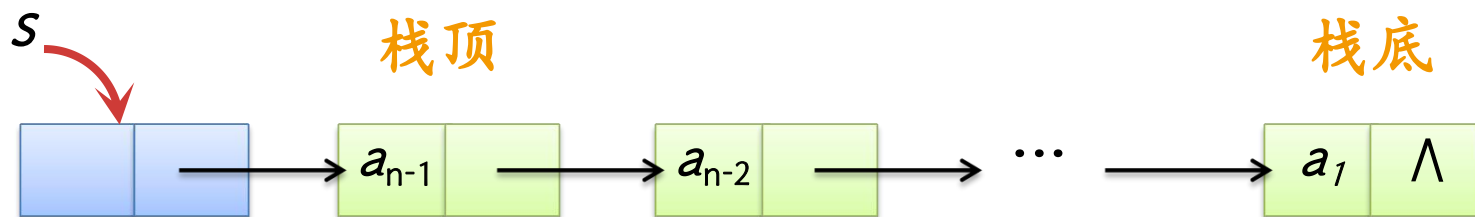
采用链表存储的栈称为**链栈**，这里采用带头结点的单链表实现。



一个**链栈**的示意图

3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现



链栈的4要素:

- 栈空条件: $s \rightarrow \text{next} == \text{NULL}$
- 栈满条件: 不考虑
- 进栈 e 操作: 将存放 e 的结点插入到头结点之后
- 退栈操作: 取出头结点之后结点的元素并删除之

3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

链栈中数据结点的类型**LinkStNode**声明如下:

```
typedef struct linknode
{   ElemType data;           //数据域
    struct linknode *next; //指针域
} LinkStNode;
```



3.1 栈

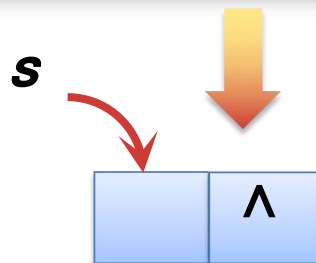
3.1.3 栈的链式存储结构及其基本运算的实现

在链栈中，栈的基本运算算法如下。

(1) 初始化栈initStack(&s)

建立一个空栈 s 。实际上是创建链栈的头结点，并将其next域置为NULL。

```
void InitStack(LinkStNode *&s)
{ s=(LinkStNode *)malloc(sizeof(LinkStNode));
  s->next=NULL;
}
```

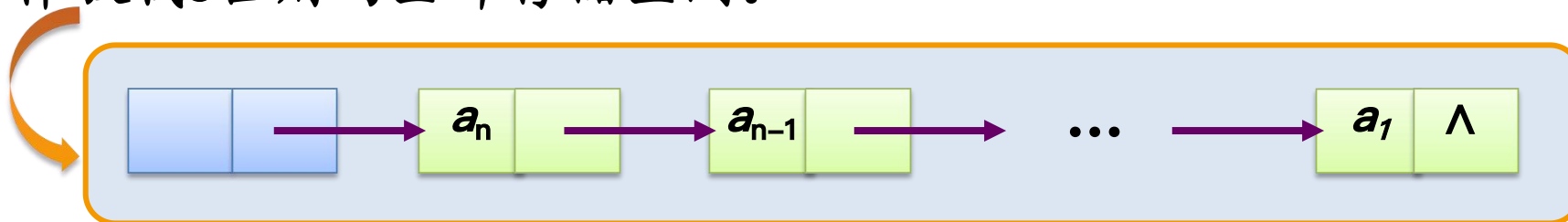


3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

(2) 销毁栈DestroyStack(&s)

释放栈s占用的全部存储空间。



```
void DestroyStack(LinkStNode *&s)
{   LinkStNode *p=s, *q=s->next;
    while (q!=NULL)
    {   free(p);
        p=q;
        q=p->next;
    }
    free(p);    //此时p指向尾结点，释放其空间
}
```

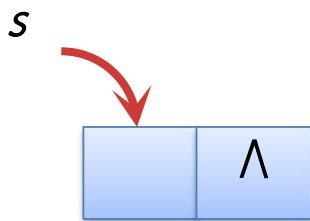
3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

(3) 判断栈是否为空 StackEmpty(s)

栈S为空的条件是 $s \rightarrow next == NULL$ ，即单链表中没有数据结点。

```
bool StackEmpty(LinkStNode *s)
{
    return(s->next==NULL);
}
```



空栈的情况

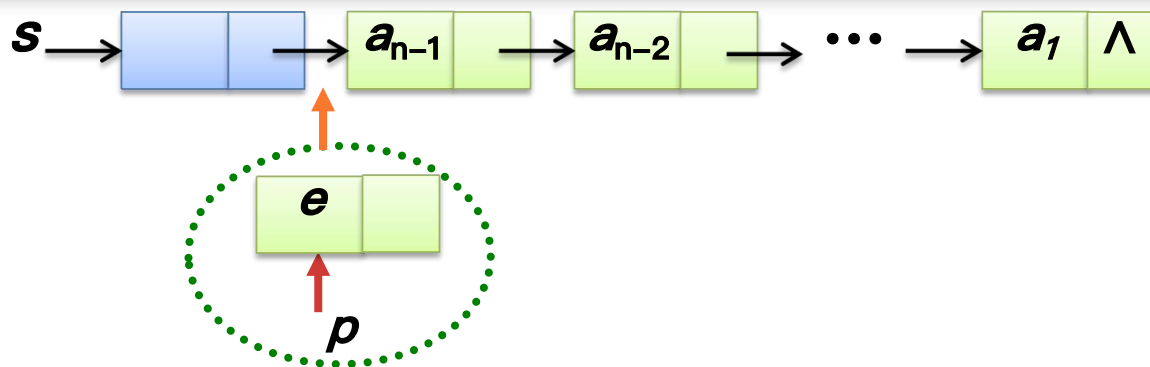
3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

(4) 进栈 $\text{Push}(\&s, e)$

将新数据结点插入到头结点之后。

```
void Push(LinkStNode *&s, ElemType e)
{ LinkStNode *p;
  p=(LinkStNode *)malloc(sizeof(LinkStNode));
  p->data=e;           //新建元素e对应的结点p
  p->next=s->next;     //插入p结点作为开始结点
  s->next=p;
}
```



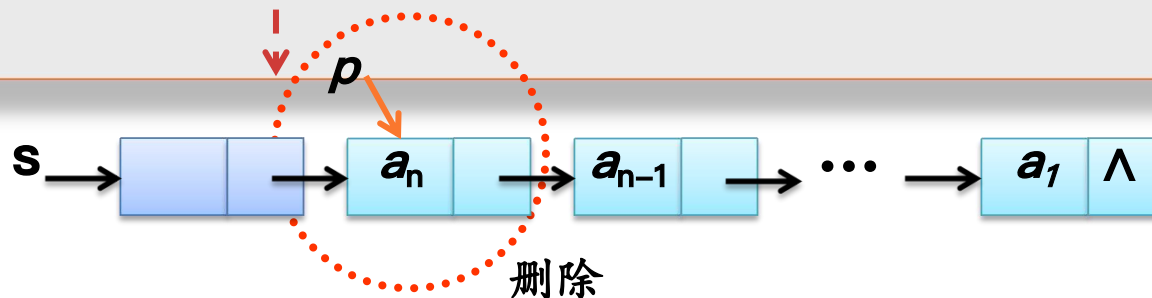
3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

(5) 出栈Pop(&s, &e)

栈非空时，将头结点后继数据结点的数据域赋给e，然后将其删除。

```
bool Pop(LinkStNode *&s, ElemType &e)
{ LinkStNode *p;
  if (s->next==NULL)           //栈空的情况
    return false;
  p=s->next;                    //p指向开始结点
  e=p->data;
  s->next=p->next;              //删除p结点
  free(p);                     //释放p结点
  return true;                  //返回真 表示操作成功
}
```



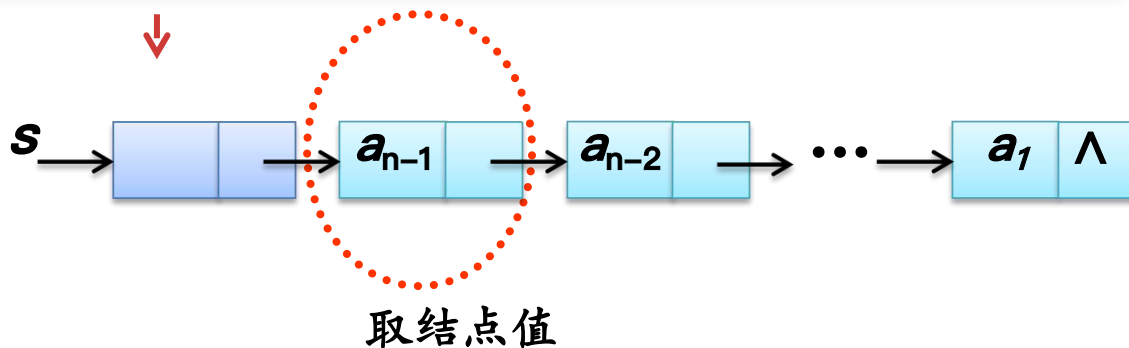
3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

(6) 取栈顶元素GetTop(s, e)

在栈不为空的条件下，将头结点后继数据结点的数据域赋给e。

```
bool GetTop(LinkStNode *s, ElemType &e)
{ if (s->next==NULL)           //栈空的情况
  return false;
  e=s->next->data;
  return true;
}
```



3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

链栈的应用算法设计

【例3.5】 编写一个算法判断输入的表达式中括号是否配对（假设只含有左、右圆括号）。

算法设计思路

一个表达式中的左右括号是按最近位置配对的。所以利用一个栈来进行求解。这里采用链栈。

3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

表达式括号**不配对**情况的演示

例如: exp= “(())”

↑ ↑ ↑ ↑ ↑

(
(

① ‘(’ 进栈

② ‘(’ 进栈

③ 遇到’)’, 栈顶为’(’, 退栈

④ 遇到’)’, 栈顶为’(’, 退栈

⑤ 遇到’)’, 栈为空, 返回false

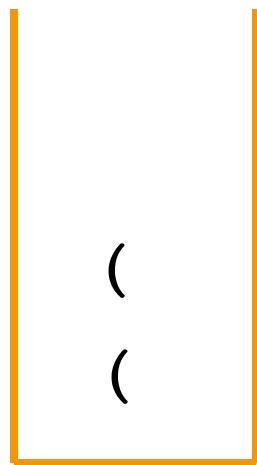


3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

表达式括号配对情况的演示

例如: exp= “(())”



① ‘(’ 进栈

② ‘(’ 进栈

③ 遇到’)’，栈顶为’ (’，退栈

④ 遇到’)’，栈顶为’ (’，退栈

栈空且exp扫描完，返回true



3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

```
bool Match(char exp[], int n)
{
    int i=0; char e;
    bool match=true;
    LinkStNode *st;
    InitStack(st);           //初始化栈
    while (i<n && match)     //扫描exp中所有字符
    {
        if (exp[i]=='(')
            Push(st, exp[i]);
    }
```

3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

```
else if (exp[i]=='')           //当前字符为右括号
{ if (GetTop(st, e)==true)
    { if (e!='(')              //栈顶元素不为'('时不匹配
        match=false;
      else
        Pop(st, e);           //将栈顶元素出栈
    }
    else match=false;         //无法取栈顶元素时不匹
    配
  }
  i++;                         //继续处理其他字符
}
```

3.1 栈

3.1.3 栈的链式存储结构及其基本运算的实现

```
if (!StackEmpty(st))  
    match=false;  
DestroyStack(st);           //销毁栈  
return match;  
}
```

注意：只有在表达式扫描完毕且栈空时返回true。

3.1 栈

3.1.4 栈的应用

栈和队列都是存放多个数据的容器。通常用于存放临时数据：

- 如果后放入的数据先处理，则使用栈。
- 如果先放入的数据先处理，则使用队列。

3.1 栈

3.1.4 栈的应用

1. 简单表达式求值

➤ 问题描述

这里限定的简单表达式求值问题是：用户输入一个包含“+”、“-”、“*”、“/”、正整数和圆括号的合法算术表达式，计算该表达式的运算结果。

3.1 栈

3.1.4 栈的应用

1. 简单表达式求值

➤ 数据组织

简单表达式采用字符数组exp表示，其中只含有“+”、“-”、“*”、“/”、正整数和圆括号。

为了方便，假设是合法的算术表达式，例如，

exp= “1+2*(4+12)”；

在设计相关算法中用到栈，这里采用顺序栈存储结构。

3.1 栈

3.1.4 栈的应用

1. 简单表达式求值

➤ 运算算法设计

运算符位于两个操作数中间的表达式称为 **中缀表达式**。例如，
"1+2*3"就是一个中缀表达式。

中缀表达式的运算规则：“先乘除，后加减，从左到右计算，先括号内，后括号外”。

因此，中缀表达式不仅要依赖运算符优先级，而且还要处理括号。

3.1 栈

3.1.4 栈的应用

1. 简单表达式求值

➤ 运算算法设计

算术表达式的另一种形式是**后缀表达式**或**逆波兰表达式**，就是在算术表达式中，运算符在操作数的后面。

如"1+2*3"的后缀表达式为"1 2 3 * +".

后缀表达式特点:

- 没有括号，已考虑了运算符的优先级。
- 只有操作数和运算符，而且越放在前面的运算符来越优先执行。

3.1 栈

3.1.4 栈的应用

1. 简单表达式求值

➤ 运算算法设计

在算术表达式中，如果运算符在操作数的前面，称为前缀表达式，如"1+2*3"的前缀表达式为"+ 1 * 2 3"。

● 中缀表达式: 1 + 2 * 3

● 后缀表达式: 1 2 3 * +

● 前缀表达式: + 1 * 2 3

} 运算数的相对次序相同

3.1 栈

3.1.4 栈的应用

1. 简单表达式求值

中缀表达式的求值过程：

- 将中缀算术表达式转换成后缀表达式。
- 对该后缀表达式求值。

3.1 栈

3.1.4 栈的应用

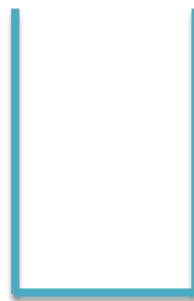
1. 简单表达式求值

(1) 将算术表达式转换成后缀表达式

exp $\Rightarrow$ **postexp**

扫描**exp**的所有字符:

- 数字字符直接放在postexp中
- 运算符通过一个栈来处理优先级



运算符栈



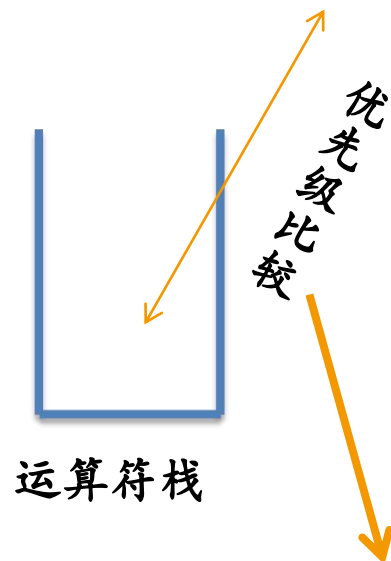
3.1 栈

3.1.4 栈的应用

$\text{exp} \Rightarrow \text{postexp}$

◆ 情况1 (没有括号)

$\text{exp} =$ “1 + 2 + 3”



$\text{postexp}:$

“1+2+3” $\Rightarrow$ “1 2 + 3 +”

- 优先级相同：先进栈的先退栈即先执行,只有大于栈顶优先级才能直接进栈
- exp 扫描完毕,所有运算符退栈

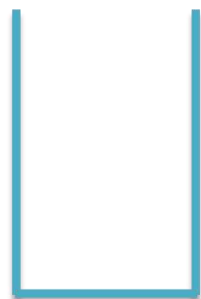
3.1 栈

3.1.4 栈的应用

exp $\Rightarrow$ postexp

◆ 情况2 (带有括号)

exp= “ 2 * (1 + 3) - 4 ”



运算符栈

postexp:



postexp = “2 1 3 + * 4 -”

- 开始时, 任何运算符都进栈
- (: 一个子表达式开始, 进栈
- 栈顶为(: 任何运算符进栈
-): 退栈到 (
- 只有大于栈顶的优先级, 才进栈; 否则退栈

3.1 栈

3.1.4 栈的应用

中缀表达式: $(56-20)/(4+2)$

后缀表达式: $56\ 20\ -\ 4\ 2\ +\ /\$



3.1 栈

3.1.4 后缀表达式转中缀表达式

规则:遇到数字就进栈,遇到符号将处于栈顶两个数字出栈,进行运算,并将结果进栈,直至最终获得结果。

后缀表达式= “2 1 3 + * 4 -”

运算结果: 4



3.1 栈

3.1.4 后缀表达式转中缀表达式

后缀表达式: **9 3 1 - 3 * + 10 2 / +**

运算结果: **20**

